# Design Document

## ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

**Course:** COEN838 — Advanced Software Engineering
**Student:** Idris Muhammad Abubakar
**Matric No:** P25EGCP9003

---

## 1. Architectural Overview

### 1.1 Architectural Style

ABU-SSO is a **modular monolith** (the Identity Provider) **plus six independent microservices** that trust its JWTs.

**Why modular monolith for the IdP (and not microservices from day one)?**

| Consideration | Modular Monolith | Microservices |
|---|---|---|
| Team size | ✅ 1 developer | ❌ Needs 3+ teams |
| Deployment complexity | ✅ Single process | ❌ Container orchestration |
| Learning curve | ✅ Simple | ❌ High |
| Growth path | ✅ Extractable | ✅ Native |
| Consistency | ✅ Single transaction | ❌ Saga needed |

The IdP is built by one developer, so a monolith with clear feature boundaries is the right trade-off. However, the service boundaries are **already drawn** — each router (`auth`, `wallet`, `id_card`, `admin`) could be extracted into its own service without rewriting business logic.

**The distributed layer was subsequently added**: six independent mock services (Portal, LMS, Library, Hostel, Wi-Fi, Result Checker) demonstrate real SSO across process boundaries. See Section 8.

### 1.2 Layer Responsibilities (IdP)

┌──────────────────────────────────────────────────────────────┐ │ Presentation │ Jinja2 HTML + CSS + JS │ (app/templates, app/static) │ ├──────────────────────────────────────────────────────────────┤ │ HTTP / Routers │ FastAPI route handlers │ (app/routers/.py) │ (thin — no business logic) ├──────────────────────────────────────────────────────────────┤ │ Services │ Business rules │ (app/services/.py) │ (no HTTP, no DB session creation) ├──────────────────────────────────────────────────────────────┤ │ Models & Schemas │ SQLAlchemy ORM + Pydantic │ (app/models, app/schemas) │ ├──────────────────────────────────────────────────────────────┤ │ Persistence │ SQLite (dev) / PostgreSQL (prod) │ (app/database.py) │ └──────────────────────────────────────────────────────────────┘

**Rule enforced:** each layer only talks to the one directly below it. Routers never touch the database directly — they call services.

### 1.3 Inter-Component Communication

- **Client → IdP:** HTTP/1.1 with JSON bodies (or HTML for pages).
- **IdP → Third-party services:** JWT forwarded in `Authorization: Bearer`.
- **Service → IdP (token verification):** synchronous REST call to `/auth/me`.
- **Style chosen:** synchronous request-response, because authentication decisions must be immediate.

Future asynchronous events (e.g., "notify user on wallet top-up") would use a message broker — out of scope for this prototype.

---

## 2. Component Design

### 2.1 Authentication Component

| File | Responsibility |
|---|---|
| `app/routers/auth.py` | Endpoints: `/auth/register` (bootstrap), `/auth/login`, `/auth/me` |
| `app/services/user_service.py` | `create_user`, `authenticate_user`, lookup helpers |
| `app/utils/security.py` | `hash_password`, `verify_password` (bcrypt) |
| `app/utils/jwt.py` | `create_access_token`, `decode_token`, `get_current_user` dependency |
| `app/utils/rbac.py` | `require_role(*roles)` — dependency factory for role gating |
| `app/schemas/user.py` | `UserCreate`, `UserRead`, `UserLogin`, `Token` |

**Key design decision #1:** `get_current_user` is a FastAPI dependency. Any protected route declares `current_user: User = Depends(get_current_user)` — the authentication logic lives in exactly one place.

**Key design decision #2 — bootstrap-only registration:**

```
@router.post("/register")
def register(payload, db):
    if db.query(User).count() == 0:
        # First-ever user on a fresh DB → admin bootstrap
        return create_user(db, ..., role="admin")
    if settings.allow_public_registration:  # tests only
        return create_user(db, ..., role=payload.role)
    # Production: refuse
    raise HTTPException(403, "Public registration is disabled.")

    This closes the self-registration vulnerability discovered during development.
Real ABU students don't self-register — ICT creates accounts after admission.

Login is by matric/staff number, not email. authenticate_user(db, matric_no,
password) normalises and looks up by matric. Email is stored as a field for
```

```
records but is not used for authentication.

2.2 Wallet Component
File    Responsibility
app/routers/wallet.py   Endpoints: /wallet, /wallet/topup, /wallet/pay,
/wallet/transactions
app/services/wallet_service.py  get_or_create_wallet, top_up, pay,
list_transactions
app/models/wallet.py    Wallet + Transaction ORM models
app/schemas/wallet.py   WalletRead, TransactionRead, TopUpRequest, PayRequest

Key design decision: every wallet operation is atomic — the balance update and the
transaction insert happen in a single db.commit(). If the balance is insufficient,
pay() raises before touching the DB.

2.3 Digital ID Card Component
File    Responsibility
app/routers/id_card.py  Endpoint /id-card
app/services/qr_service.py  build_id_card_payload, generate_qr_data_url
app/schemas/id_card.py  IdCardResponse
Key design decision: the QR code encodes only {id, matric_no, role} — no personal
data — reducing the risk of information disclosure if a card is photographed.

2.4 Admin Component (RBAC)
File    Responsibility
app/routers/admin.py    Endpoints: list/create users, role change,
activate/deactivate, top up wallet, stats
app/services/admin_service.py   User listing, role changes, admin top-ups, stats
aggregation
app/schemas/admin.py    UserAdminView, AdminTopUpRequest, AdminRoleUpdateRequest,
AdminStats
app/utils/rbac.py   require_role("admin")

Key design decision: the entire admin router declares dependencies=
[Depends(require_role("admin"))] — every endpoint in the router is automatically
protected. Non-admin callers receive HTTP 403 Forbidden.

POST /admin/users is the production path for account creation. After the first
bootstrap admin, this is the only way to create new users.

2.5 Presentation Component
File    Responsibility
app/routers/pages.py    Serves /login, /dashboard, /admin
app/templates/base.html, login.html, dashboard.html, admin.html Jinja2 templates
app/static/css/style.css    Green campus theme
app/static/js/app.js    Shared JS; page logic inline in templates
The dashboard calls /auth/me, /wallet, /wallet/transactions, and /id-card using
the JWT in localStorage. It also wires six service cards to their respective mock-
service URLs (auto-detects local vs production).

3. Data Design
3.1 Entity Relationship Diagram
See docs/diagrams/class_model.png.
```

```
3.2 Table Definitions
users

Column  Type    Constraint
id  VARCHAR(36) PRIMARY KEY
email   VARCHAR(255)    UNIQUE, NOT NULL, INDEXED
password_hash   VARCHAR(255)    NOT NULL
full_name   VARCHAR(255)    NULL
role    VARCHAR(50) NOT NULL, default 'student'
matric_no   VARCHAR(50) UNIQUE, NOT NULL, INDEXED
is_active   BOOLEAN NOT NULL, default TRUE
created_at  DATETIME    NOT NULL
Note: matric_no is now UNIQUE and NOT NULL because it's the login identifier.

wallets

Column  Type    Constraint
id  VARCHAR(36) PRIMARY KEY
user_id VARCHAR(36) FOREIGN KEY → users.id, UNIQUE
balance NUMERIC(10,2)   NOT NULL, default 0
created_at  DATETIME    NOT NULL

transactions

Column  Type    Constraint
id  VARCHAR(36) PRIMARY KEY
wallet_id   VARCHAR(36) FOREIGN KEY → wallets.id
amount  NUMERIC(10,2)   NOT NULL
type    VARCHAR(20) NOT NULL, credit or debit
description VARCHAR(255)    NULL
created_at  DATETIME    NOT NULL

3.3 Design Decisions
UUIDs instead of integers for primary keys — safe to expose in URLs, work
identically in SQLite and PostgreSQL.

Numeric(10,2) for money — avoids floating-point rounding errors.

Wallet created lazily — users who never use the wallet don't consume a row.

Cascade delete — deleting a wallet removes its transactions automatically.

matric_no unique + indexed — because it's the login identifier.

Case-insensitive matric matching — matric is stored uppercase; lookups normalise
input.

4. Sequence Design — Payment Flow
See docs/diagrams/sequence_login.png for the login sequence. Below is the wallet
payment flow:
Client (browser)
   │
   │  POST /wallet/pay {amount: 250}
   │  Authorization: Bearer <JWT>
```

```
   ▼
Router (wallet.py: pay_from_wallet)
   │
   ├─ Depends(get_current_user) ──► JWT validated, User loaded
   │
   ├─ Depends(get_db) ──────────────► DB session opened
   │
   ├─ wallet_service.get_or_create_wallet(db, user)
   │
   └─ wallet_service.pay(db, wallet, 250)
        │
        ├─ if amount <= 0: raise ValueError
        ├─ if balance < amount: raise ValueError
        │
        ├─ wallet.balance -= 250
        ├─ db.add(Transaction(type='debit', amount=250))
        ├─ db.commit()
        │
        └─ return Transaction
   │
   ▼
Client receives 201 Created + transaction JSON
```

## 5. Resilience & Scalability

### 5.1 Resilience

Fail securely: auth failures return 401 with a generic message; unauthorized roles return 403.

Transaction atomicity: wallet balance and transaction record commit together.

Stateless auth: if any server instance dies, the user's token works on another instance.

Bootstrap safety: a fresh deployment can always be initialised (first user → admin); a compromised public register endpoint cannot create new admins.

Mock services degrade gracefully: if the IdP is unreachable, they return HTTP 503 instead of serving unauthenticated content.

### 5.2 Scalability

Horizontal scaling: stateless JWT — multiple IdP instances behind a load balancer.

Mock services scale independently — each runs on its own Render web service.

Database growth: SQLite is single-writer; at production scale, swap DATABASE_URL to PostgreSQL — no code changes.

Read replicas: future work can add PostgreSQL read replicas for /auth/me.

### 5.3 Known Limitations

No rate limiting yet (planned via slowapi).

No refresh tokens — users must re-login every 15 minutes.

No async task queue — everything is synchronous in the request cycle.

Mock services use URL query-param token passing; production would use HttpOnly cookies or BFF pattern.

Render free tier sleeps after 15 minutes of inactivity.

6. Design Patterns Applied
Pattern Where Why
Repository / Service Layer app/services/*.py Business logic separated from HTTP
Dependency Injection FastAPI Depends() Testable; single source of truth
Schema / DTO Pydantic *Create, *Read classes Never leak ORM internals
Singleton settings = Settings() in config.py One configuration source
Factory create_engine, sessionmaker in database.py Configurable DB backend
Factory (dependency) require_role(*roles) in rbac.py Reusable role gating
Middleware / Decorator get_current_user dependency Cross-cutting auth
Layered Architecture Routers / Services / Models Clear responsibilities
Service-Oriented mock-services/*/main.py Independent deployable units
Bootstrap Pattern auth.register First-user admin creation on fresh DB

7. Technology Choices — Justification
Choice Alternative Why we chose ours
FastAPI Flask / Django Async-native, auto-generated OpenAPI, type-driven
SQLAlchemy Raw SQL / Django ORM Portable between SQLite and PostgreSQL
SQLite (dev) PostgreSQL from day 1 Zero setup for a beginner; migrate later
JWT (HS256) Server-side sessions Stateless; ideal for SSO across services
bcrypt SHA-256 Designed for password storage; salted; slow by design
RBAC middleware Hard-coded role checks Reusable, testable, single source of truth
Jinja2 + vanilla JS React / Vue Simpler; sufficient for three-page UI
pytest unittest Cleaner syntax, better fixtures
Playwright Selenium Faster, modern, first-class Python support
StaticPool (tests) Fresh DB per test One in-memory DB shared by all sessions
GitHub Actions Jenkins Free, integrated with GitHub
Bandit Manual review Automated SAST — evidence for security rubric
Render + Neon Single-provider hosting Free tier stability; separation of compute and storage

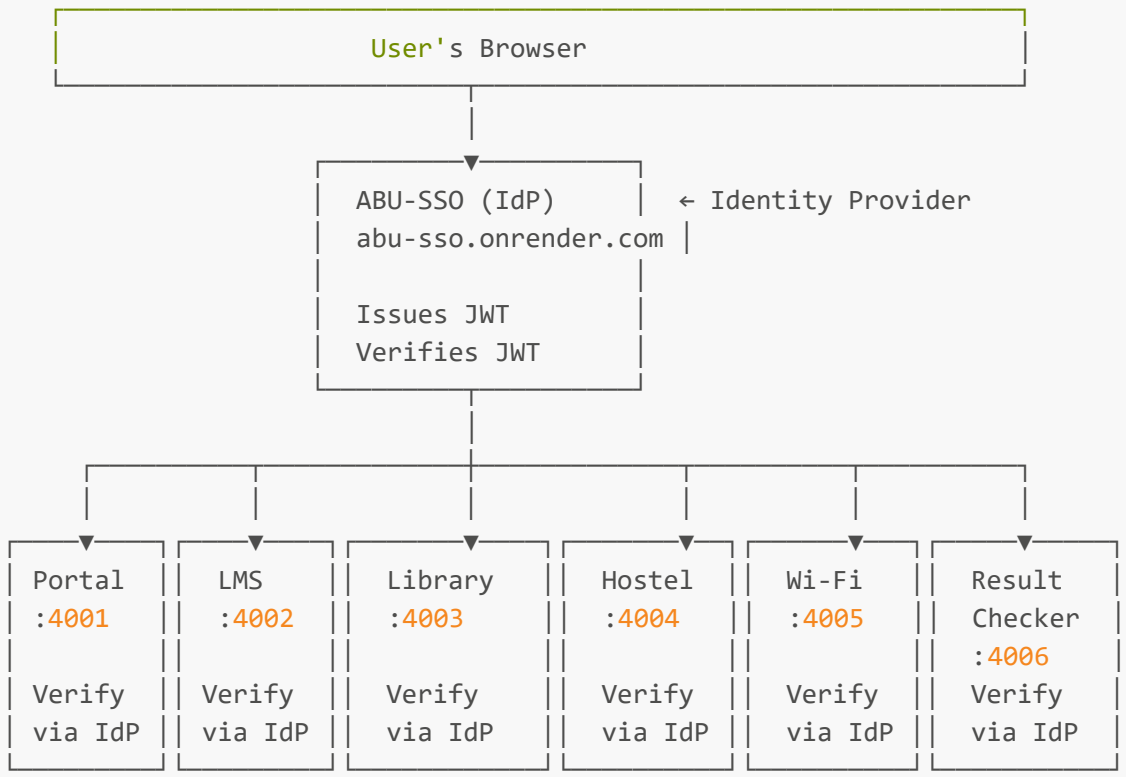
8. Distributed SSO Architecture (Six Mock Services)
To demonstrate that ABU-SSO is genuinely a distributed, service-oriented system (not merely a login page), six independent mock services were built that trust ABU-SSO as their identity provider.

8.1 Why Six Services?
The original problem statement (COEN838 Question 7) says:

"Students currently juggle multiple credentials: the central student portal for course registration, the LMS (Moodle) for classes, the Wi-Fi portal, and physical ID cards for hostel/library access."

The problem lists six distinct systems. Building six separate services that independently verify the SSO token proves the core claim: one login → many services.

8.2 Service Topology




In production (Render): each service is deployed at its own HTTPS URL (abu-sso-portal.onrender.com, etc.).
In development (localhost): each runs on a distinct port.

8.3 Design Decisions

| Decision | Choice | Justification |
|---|---|---|
| Service boundary | Each service is a standalone FastAPI app | Independence — one can crash without affecting others |
| Data ownership | No service has a user table IdP is single source of truth; services are stateless verifiers | |
| Token verification | HTTP call to IdP/auth/me | Simpler than sharing the JWT secret; models OIDC UserInfo |
| Token transport | URL query param ?token=<JWT> | Simplest cross-origin demo; production would use cookies or BFF |
| Coupling | Loose — services know only the IdP's base URL (env var) | Swap or scale IdP without touching services |
| Failure mode | HTTP 503 if IdP unreachable | Fail-secure: never serve data without verification |
| Environment detection | Dashboard JS picks URLs based on window.location.hostname | Same code works locally and on Render |

8.4 Inter-Service Communication Patterns
Pattern used: Synchronous REST with token-based trust.

Each service performs GET {IDP_URL}/auth/me with the JWT in the Authorization header.

The IDP_URL is an environment variable on Render, set to the main app's URL.

Response is used for that request only (not persisted).

Latency cost: one extra HTTP round-trip (~5–10 ms local; 100–200 ms to Render).

```
Alternative patterns considered:

Pattern Why we didn't choose it
Shared JWT secret (each service verifies signature) Requires distributing secret; harder to rotate
RS256 public-key verification   Better for production; overkill for demo
Event-based (Kafka/RabbitMQ)    Correct for async workloads; unnecessary for synchronous reads
OIDC Discovery + JWKS   Production-grade; requires HTTPS and DNS setup beyond demo scope

8.5 Scalability & Resilience Implications
Horizontal scale-out — services are stateless; N instances can run behind a load balancer.

IdP is the bottleneck — every request from every service calls /auth/me. In production, this would be mitigated by (a) caching verified tokens at the service for their remaining TTL, and (b) using RS256 with JWKS for local verification.

Failure isolation — if the Library service dies, Portal and LMS remain fully functional.

Fail-secure — if the IdP is unreachable, services return 503, never serve data unauthenticated.

8.6 How It Maps to the Rubric
Rubric line How the mock services satisfy it
"distributed/service-oriented solution" Six independent services, each a separate process/deployment
"inter-component communication pattern" Synchronous REST; token via HTTP header; documented above
"scalability"   Stateless services; IdP bottleneck documented with mitigations
"resilience"    Fail-secure when IdP unreachable; failure isolation

8.7 Files
File    Port    Purpose
mock-services/portal/main.py    4001    Course registration (Student Portal)
mock-services/lms/main.py   4002    Lectures & assignments (Moodle)
mock-services/library/main.py   4003    Books & fines
mock-services/hostel/main.py    4004    Room booking & gate access
mock-services/wifi/main.py  4005    Network access & device registration
mock-services/results/main.py   4006    Semester results & GPA
mock-services/*/templates/service.html  —   Shared template (copied to all six)
mock-services/*/requirements.txt    —   Independent dependency files for deployment

8.8 Demo Flow
User logs in once at ABU-SSO (/login).

Dashboard renders six service cards.

Clicking Student Portal opens https://abu-sso-portal.onrender.com/?token=<JWT>.
```

The Portal service:

Extracts the token from the query string.

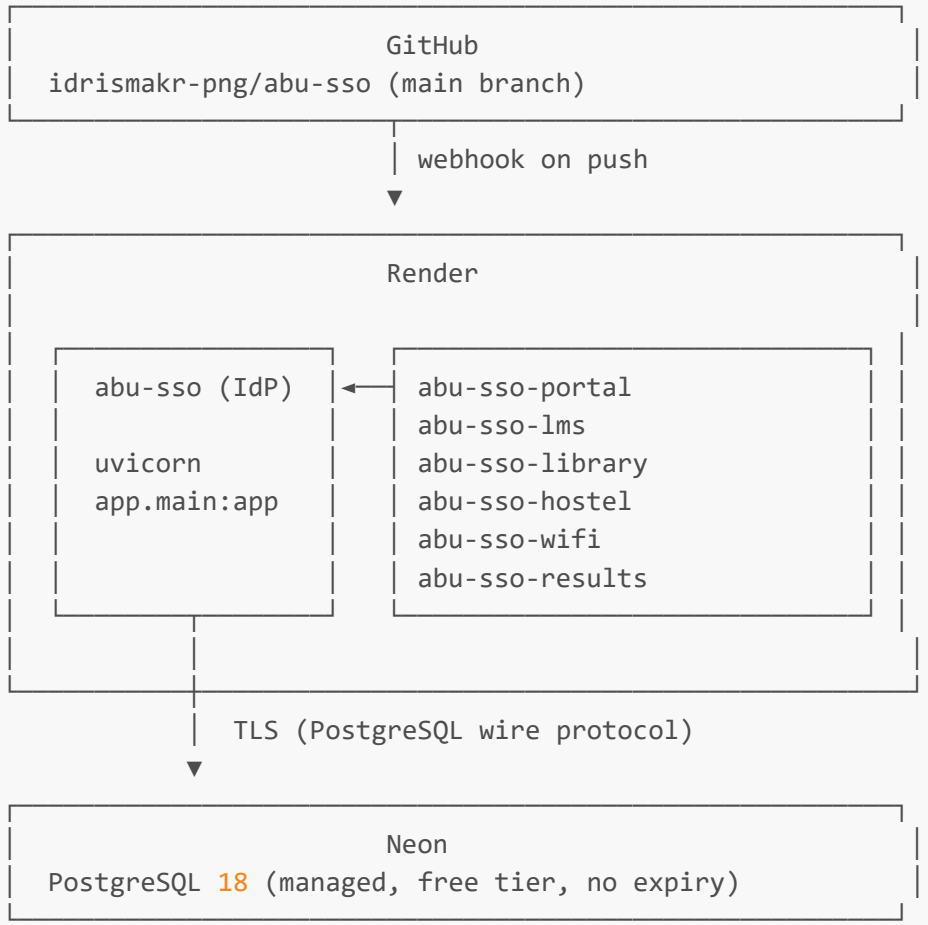
Calls GET https://abu-sso.onrender.com/auth/me.

Renders the Portal page with the user's profile.

Same flow for LMS, Library, Hostel, Wi-Fi, Result Checker — no second password prompt in any of them.

This demonstrates the single sign-on guarantee end-to-end, publicly, on the internet.

9. Deployment Architecture




Deployment triggers: every push to main fires a GitHub webhook → Render detects the change → rebuilds the affected services → swaps them into production with zero downtime.

Configuration: render.yaml declares all seven web services. Environment variables (DATABASE_URL, JWT_SECRET, IDP_URL) are managed in Render's dashboard.